Giới thiệu về RE

(dịch ngược)

Dịch xuôi

- Lập trình viên viết chương trình ở ngôn ngữ bậc cao (C/C++, C#, Java, ...)
- Máy tính chỉ hiểu được ngôn ngữ máy
- Cần biên dịch từ ngôn ngữ lập trình bậc cao sang ngôn ngữ máy để máy tính có thể chạy được
- Sử dụng *trình biên dịch* (compiler) để dịch sang mã máy
- Thu được file chạy (.exe)

Dịch xuôi

- Bài toán: từ file .exe, liệu có thể thu được mã nguồn ở ngôn ngữ bậc cao hay không?
- Từ mã máy có thể chuyển trực tiếp về dạng hợp ngữ được, tuy nhiên khó có thể đọc hiểu được với chương trình phức tạp
- Vấn đề:
  - Tên hàm, tên biến sẽ bị mất đi (máy tính không cần biết đến tên biến mà chỉ cần địa chỉ)
  - Một số cấu trúc bậc cao bị mất đi (cấu trúc hướng đối tượng, kiểu của biến, ...)

Dịch ngược

- *Dịch ngược* (reverse engineering) là một ngành nhằm giải quyết bài toán này
- Tìm lại *thiết kế* (design) từ *sản phẩm* (product)
- Phục vụ nhiều mục đích:
  - Giao tiếp với các phần mềm khác (Samba, Wine, OpenOffice, ...)
  - Phân tích hành vi mã độc
  - Nghiên cứu, học hỏi
- Điều khoản sản phẩm của một số doanh nghiệp không cho phép điều này

Dịch ngược

- Dựa trên nhận xét: mọi chương trình muốn chạy đều phải chạy trên CPU => luôn có thể theo dõi quá trình chạy của chương trình nếu có phép của hệ điều hành
- Dựa vào đối tượng cần dịch ngược và mục tiêu mà phương pháp cụ thể có thể khác nhau

Hướng tiếp cận

- Có 2 hướng tiếp cận chính:
  - Phân tích tĩnh (static analysis): sử dụng các công cụ phân tích tĩnh để phân tích trực tiếp trên file chạy nhưng không chạy chương trình đó
  - Phân tích động (dynamic analysis): sử dụng trình gỡ lỗi (debugger) và các công cụ theo dõi (monitor) để theo dõi hành vi chương trình trong quá trình chạy
- Trong thực tế, thường sử dụng 2 hướng tiếp cận này phối hợp lẫn nhau

Static analysis

- Phương pháp:
  - Tìm các dữ liệu có sẵn trong chương trình
  - Đọc hiểu và phân tích luồng chương trình
  - Dự đoán luồng chương trình và quan hệ giữa các dữ liệu
  - Từ đó khôi phục được logic ban đầu
- Công cụ: IDA Pro, Binary Ninja, radare2, RetDec, ...

IDA (Interactive Disassembler)

- Là một công cụ phân tích tĩnh rất mạnh, đa nền tảng
- Tích hợp tính năng gỡ lỗi để phân tích động
- Trực quan, dễ nhìn
- Có bản miễn phí (đến phiên bản 7.0 trong thời điểm viết bài)

IDA (Interactive Disassembler)

IDA -

File Edit Jump Search View Debugger Options Windows Help

No debugger

Library function | Data | Regular function | Unexplored | Instruction | External symbol

Functions wind... | IDA View-A | Hex View-1 | Structures | Enums | Imports | Exports

Function name

- SendDlgItemMessageW
- SetDlgItemInt
- sub_180001100
- DdeUninitialize
- sub_1800012D4
- sub_180001344
- sub_1800013B8
- sub_1800013EC

Line 21 of 2358

Graph overview

```
sub_180001A98 proc near
push    rbx
sub     rsp, 20h
lea     rcx, unk_1800A3230
xor     ebx, ebx
call    cs:RtlEnterCriticalSection
cmp     cs:dword_1800A3074, ebx
jle     short loc_180001AE5
```

```
loc_180001AE5:
```

100.00% (-53,-30) (22,0) 00000EBF 0000000180001ABF: sub_180001A98:loc_ (Synchronized with Hex View-1)

Output window

The initial autoanalysis has been finished.

Python

AU: idle Down Disk: 101GB

Hex-Rays Decompiler

- Là một phần mở rộng của IDA Pro
- Có khả năng dịch ngược từ hợp ngữ về ngôn ngữ giả C (nếu hợp ngữ sinh ra tuân theo đúng quy chuẩn của trình biên dịch)
- Cho kết quả rất tốt
- Chi phí cao ($2629 ở thời điểm viết bài)

Hex-Rays Decompiler

IDA -

File Edit Jump Search View Debugger Options Windows Help

No debugger

Library function | Data | Regular function | Unexplored | Instruction | External symbol

Functions wind... | IDA Vi... | Pseudoco... | Hex Vi... | Struc... | En... | Im... | Ex...

Function name

- DrawFocusRect
- sub_180001760
- USER32_2568
- SwitchToThisWindow
- USER32_2560
- DdeFreeStringHandle
- GetPointerInfo
- sub_180001A98

Line 21 of 2358

Graph overview

```
1  signed __int64 sub_180001A98()
2  {
3    __int64 v0; // rbx@1
4
5    v0 = 0i64;
6    RtlEnterCriticalSection(&unk_1800A3230);
7    if ( dword_1800A3074 <= 0 )
8    {
9      if ( dword_1800A3144 == 1 )
10     {
11       *(&off_1800A2010 + 1) = 0i64;
12       LODWORD(Size) = 0;
13       v0 = qword_1800A3078;
14       qword_1800A3078 = 0i64;
15       dword_1800A3144 = 0;
16       *(&off_1800A2000 + 1) = WINNLSGetIMEHotkey;
17       off_1800A2010 = SetDebugErrorLevel;
18       off_1800A2000 = 40;
19     }
20   }
```

00000F18 sub_180001A98:16

Output window

```
1800A3144: using guessed type int dword_1800A3144;
```

Python

AU: idle | Down | Disk: 101GB

Phân tích động

- Phương pháp:
  - Theo dõi hành vi chạy của chương trình
  - Kiểm tra xem hành vi chạy có gì bất thường hay không
  - Theo dõi tại những điểm đáng nghi trong chương trình
- Công cụ: x64dbg, IDA Pro, OllyDbg, gdb, Procmon, ...

OllyDbg

- Là một công cụ sử dụng để gỡ lỗi các chương trình Windows 32 bit, có từ những năm 2000
- Hoàn toàn miễn phí
- Tiện lợi, dễ dùng
- Có cộng đồng người dùng lớn, nhiều plugin hay & nhiều bài hướng dẫn
- Dễ dàng thêm các tính năng

OllyDbg

OllyDbg - DeSmuME_VS2008_NoConsole.exe - [CPU - main thread, module ntdll]

File View Debug Plugins Options Window Help

```
77C50CC7  74 0E          JE SHORT ntdll.77C50CD7
77C50CC9  8B0D A487CF77  MOV ECX,DWORD PTR DS:[77CF87A4]
77C50CCF  FF15 D091CF77  CALL DWORD PTR DS:[77CF91D0]      ntdll.Rt
77C50CD5  FFE1           JMP ECX
77C50CD7  894424 04      MOV DWORD PTR SS:[ESP+4],EAX
77C50CDB  895C24 08      MOV DWORD PTR SS:[ESP+8],EBX
77C50CDF  E9 483DFFFF    JMP ntdll.77C44A2C
77C50CE4  8DA424 00000000 LEA ESP,DWORD PTR SS:[ESP]
77C50CEB  EB 03          JMP SHORT ntdll.KiFastSystemCall
77C50CED  CC             INT3
77C50CEE  CC             INT3
77C50CEF  CC             INT3
77C50CF0  8BD4           MOV EDX,ESP
77C50CF2  0F34           SYSENTER
77C50CF4  8DA424 00000000 LEA ESP,DWORD PTR SS:[ESP]
77C50CFB  EB 03          JMP SHORT ntdll.KiFastSystemCallRet
77C50CFD  CC             INT3
77C50CFE  CC             INT3
77C50CFF  CC             INT3
77C50D00  C3             RETN
77C50D01  8DA424 00000000 LEA ESP,DWORD PTR SS:[ESP]
77C50D08  8DA424 00000000 LEA ESP,DWORD PTR SS:[ESP]
77C50D0F  90             NOP
```

```
Jump is taken
77C50CD7=ntdll.77C50CD7
```

```
Registers (FPU)
EAX 005D2BD5 DeSmuME_.<Mo
ECX 00000000
EDX 00000000
EBX 002B3000
ESP 0019FFF0
EBP 00000000
ESI 00000000
EDI 00000000

EIP 77C50CC7 ntdll.77C50C

C 0  ES 002B 32bit 0(FFFF
P 1  CS 0023 32bit 0(FFFF
A 0  SS 002B 32bit 0(FFFF
Z 1  DS 002B 32bit 0(FFFF
S 0  FS 0053 32bit 2B6000
T 0  GS 002B 32bit 0(FFFF
D 0
O 0  LastErr ERROR_ENVVAR

EFL 00000246 (NO,NB,E,BE,

ST0 empty 0.0
ST1 empty 0.0
ST2 empty 0.0
ST3 empty 0.0
ST4 empty 0.0
ST5 empty 0.0
```

```
Address   Hex dump                ASCII
00690000  6E 6F 20 63 61 6C 6C 69 no calli
00690008  6E 67 20 65 6E 76 69 72 ng envir
00690010  6F 6E 6D 65 6E 74 00 00 onment..
00690018  6E 6F 20 76 61 6C 75 65 no value
00690020  00 00 00 00 3F 00 00 00 ....?...
00690028  62 61 64 20 61 72 67 75 bad argu
00690030  6D 65 6E 74 20 23 25 64 ment #%d
00690038  20 28 25 73 29 00 00 00  (%s)...
00690040  6E 00 00 00 6D 65 74 68 n...meth
00690048  6F 64 00 00 63 61 6C 6C od..call
00690050  69 6E 67 20 27 25 73 27 ing '%s'
00690058  20 6F 6E 20 62 61 64 20  on bad
```

```
0019FFF0  00000000
0019FFF4  00000000
0019FFF8  00000000
0019FFFC  00000000
```

Single step event at ntdll.77C50CC7 - use Shift+F7/F8/F9 to pass exception to program

Paused

Patching

- Đôi khi, ta cần phải sửa một số hành vi của chương trình bằng cách thay đổi code (hành động này được gọi là patching)
  - Hành vi phá hoại
  - Kiểm tra xem có chạy đúng máy hay không
  - ...
- Chương trình sẽ được đưa lên bộ nhớ máy tính trước khi chạy
- Có thể tạm thời thay đổi code trên bộ nhớ (in-memory patching) hoặc thay đổi hoàn toàn code trên đĩa cứng

Anti-reversing

- Do tính chất của việc dịch ngược, ở nhiều nơi áp dụng các biện pháp nhằm làm khó người dịch ngược
  - Game online
  - Phần mềm bản quyền
  - Mã độc
- Các biện pháp chống dịch ngược thường phá các cách tiếp cận nêu trên
- Một chương trình có thể cài nhiều biện pháp chống dịch ngược cùng lúc

Packing

- Các chương trình sử dụng phương pháp packing thường sẽ ẩn logic thật đi
- Sử dụng một chương trình khác (loader) để gián tiếp thực thi logic thật
- Làm khó cho việc phân tích tĩnh & trong một số trường hợp, phức tạp hóa quá trình phân tích động
- Kĩ thuật khôi phục lại logic gốc của chương trình nhằm dễ phân tích gọi là *unpacking*

Obfuscating

- Để làm mất công sức phân tích logic của người dịch ngược, nhà sản xuất có thể thêm các đoạn code thừa nhằm che giấu mục đích thật của code
- Phương pháp này được gọi là *làm rối* (obfuscate) code
- Trong quá trình chạy, code chạy có thể bị biến đổi dựa theo điều kiện nào đấy, làm khó có thể lần theo luồng chạy chương trình (self-modifying code / polymorph)

Obfuscating

```
FunctionA()
{
  FuncAPart1();
  FuncAPart2();
  FuncAPart3();
}

FunctionB()
{
  FuncBPart1();
  FuncBPart2();
  FuncBPart3();
}

main()
{
  FunctionA();
  FunctionB();
}
```

```
main:
      jmp FAP1
FBP3: call FuncBPart3
      jmp end
FBP1: call FuncBPart1
      jmp FBP2
FAP2: call FuncAPart2
      jmp FAP3
FBP2: call FuncBPart2
      jmp FBP3
FAP1: call FuncAPart1
      jmp FAP2
FAP3: call FuncAPart3
      jmp FBP1
end:
```

Anti-debugging & Anti-VM

- Nhằm tránh chương trình khỏi bị phân tích động, người lập trình có thể thêm vào những đoạn code để kiểm tra xem chương trình có đang bị gỡ lỗi hoặc chạy trong máy ảo hay không
- Có một số cách để tránh khỏi phương pháp này, tuy nhiên không phải lúc nào cũng hiệu quả

Dịch ngược một số nền tảng khác

- Có một số ngôn ngữ không dịch sang mã máy mà dịch sang một ngôn ngữ trung gian (bytecode, IL, ...) để chạy trên máy ảo, ví dụ như Java, Visual Basic, các ngôn ngữ .NET...
- Các ngôn ngữ trung gian này thường để lại nhiều thông tin (metadata) về logic ở ngôn ngữ bậc cao
- Khi sử dụng đúng công cụ có thể sinh ra mã nguồn tương đối đẹp => giảm công sức dịch ngược

Dịch ngược một số nền tảng khác

- Có một số ngôn ngữ chạy luôn sử dụng trình phiên dịch (interpreter) không cần bước biên dịch, ví dụ như JavaScript, Python, PHP, ...
- Để ẩn giấu logic của các code này, người ta sử dụng phương pháp obfuscation
- Quá trình dịch ngược sẽ phân tích và làm đẹp code để dễ thấy được logic ban đầu

Demo & bài tập

- MFC crackme
- Keygenme